**Documentación Técnica - FP.CORE y FP.DATA**

🎯 **Para: Equipo de Desarrollo MVC**

📅 **Fecha: [Fecha actual]**

📌 **Proyecto: Math Riddles Game**

---

## 🏗️ Arquitectura Implementada

El proyecto sigue una **arquitectura en 3 capas** estrictamente separadas:

```
┌─────────────────────────────────────┐
│      FP.MVC (Presentación)          │
│  Controllers, Views, ViewModels     │
└──────────────┬──────────────────────┘
               │ Referencia
               ▼
┌─────────────────────────────────────┐
│      FP.CORE (Lógica de Negocio)    │
│  Services, DTOs, Interfaces         │
└──────────────┬──────────────────────┘
               │ Referencia
               ▼
┌─────────────────────────────────────┐
│      FP.DATA (Acceso a Datos)       │
│  Entities, Repositories, DbContext, Enums │
└─────────────────────────────────────┘
```

### ⚠️ Reglas de Dependencias (IMPORTANTE)

- ❌ **FP.DATA** NO referencia a nadie
- ✅ **FP.CORE** solo referencia a **FP.DATA**
- ✅ **FP.MVC** referencia a **FP.CORE** y **FP.DATA**

---

### 📦 FP.DATA - Capa de Acceso a Datos

### 📁 Estructura de Carpetas

```
FP.DATA/
├── Enums/
│   ├── DifficultyLevel.cs     (1=Beginner, 2=Intermediate, 3=Expert)
│   ├── RiddleType.cs          (1=MultipleChoice, 2=FreeText)
│   ├── PowerUpType.cs         (1=FiftyFifty, 2=Hint, 3=ExtraTime)
│   └── IQLevel.cs             (1-7: Beginner→Einstein)
│
├── Entities/
│   ├── PlayerSession.cs       (Sesión del jugador con GUID)
│   ├── Riddle.cs              (Acertijos matemáticos)
│   ├── GameAttempt.cs         (Intentos de respuesta)
│   └── PlayerPowerUp.cs       (Power-ups disponibles)
│
├── Repositories/
│   ├── Interfaces/
│   │   ├── IPlayerSessionRepository.cs
│   │   ├── IRiddleRepository.cs
│   │   ├── IGameAttemptRepository.cs
│   │   └── IPlayerPowerUpRepository.cs
│   │
│   └── Implementations/
│       ├── PlayerSessionRepository.cs
```

```
│       ├── RiddleRepository.cs
│       ├── GameAttemptRepository.cs
│       └── PlayerPowerUpRepository.cs
│
└── GameDbContext.cs          (Entity Framework 6 Context)
```

---

### 🔑 Entidades Principales

**1. PlayerSession**

Representa la sesión única de un jugador (sin login).

**Propiedades clave:**

- SessionId (GUID) - Identificador único del jugador
- TotalScore - Puntuación acumulada
- RiddlesSolved - Total de acertijos resueltos correctamente
- CurrentIQLevel (Enum) - Nivel de IQ actual basado en puntos

**Relaciones:**

- Tiene múltiples GameAttempts
- Tiene múltiples PlayerPowerUps

---

**2. Riddle**

Acertijos matemáticos del juego.

**Propiedades clave:**

- Question - Pregunta del acertijo
- CorrectAnswer - Respuesta correcta
- Option1-4 - Opciones para respuesta múltiple (nullable)
- Hint - Pista para el power-up
- Difficulty (Enum) - 1=Principiante, 2=Intermedio, 3=Experto

- Type (Enum) - 1=MultipleChoice, 2=FreeText
- TimeLimit - Tiempo límite en segundos (60, 45, 30)
- BasePoints - Puntos base (100, 200, 300)

---

**3. GameAttempt**

Registro de cada intento de respuesta.

**Propiedades clave:**

- PlayerSessionId - FK a PlayerSession
- RiddleId - FK a Riddle
- PlayerAnswer - Respuesta del jugador
- IsCorrect - Si acertó o no
- PointsEarned - Puntos ganados (base)
- BonusPoints - Bonus por rapidez (max 50)
- TimeSpent - Tiempo usado en segundos
- UsedFiftyFifty, UsedHint, UsedExtraTime - Power-ups usados

---

**4. PlayerPowerUp**

Power-ups disponibles por jugador.

**Propiedades clave:**

- PlayerSessionId - FK a PlayerSession
- PowerUpType (Enum) - Tipo de power-up
- AvailableCount - Cantidad disponible (inicialmente 3 de cada uno)

---

### 🔧 Repositorios Implementados

Todos los repositorios siguen el patrón **Repository Pattern** con:

**Métodos comunes:**

- GetById(int id) - Obtener por ID
- Add(entity) - Agregar nueva entidad
- Update(entity) - Actualizar entidad existente
- SaveChanges() - Persistir cambios en BD

**Métodos específicos importantes:**

**PlayerSessionRepository:**

csharp

```
PlayerSession GetBySessionId(Guid sessionId)
```

**RiddleRepository:**

csharp

```
Riddle GetRandomRiddleByDifficulty(DifficultyLevel difficulty, List<int> excludeIds)
List<Riddle> GetByDifficulty(DifficultyLevel difficulty)
```

**GameAttemptRepository:**

csharp

```
bool HasPlayerAttemptedRiddle(int playerSessionId, int riddleId)
List<GameAttempt> GetByPlayerSessionId(int playerSessionId)
```

**PlayerPowerUpRepository:**

csharp

```
PlayerPowerUp GetByPlayerAndType(int playerSessionId, PowerUpType powerUpType)
```

---

### 🗄️ Base de Datos

**Connection String en Web.config:**

xml

```
<connectionStrings>
  <add name="GameConnection"
```

```
    connectionString="Data Source=localhost;Initial Catalog=MathRiddlesDB;Integrated Security=True;MultipleActiveResultSets=True"
    providerName="System.Data.SqlClient" />
</connectionStrings>
```

**Tablas creadas:**

- `PlayerSessions` (45 riddles precargados)
- `Riddles`
- `GameAttempts`
- `PlayerPowerUps`

**Stored Procedures disponibles:**

- `sp_GetOrCreatePlayerSession`
- `sp_GetRandomRiddle`
- `sp_SubmitAnswer`
- `sp_UsePowerUp`
- `sp_GetPlayerPowerUps`
- `sp_GetPlayerHistory`

---

## 🎯 FP.CORE - Capa de Lógica de Negocio

### 📂 Estructura de Carpetas

```
```

```
FP.CORE/
├── DTOs/
│   ├── PlayerSessionDTO.cs
│   ├── RiddleDTO.cs
│   ├── GameAttemptDTO.cs
│   ├── PowerUpDTO.cs
│   ├── SubmitAnswerRequest.cs
│   └── SubmitAnswerResponse.cs
│
├── Interfaces/
│   ├── IPlayerSessionService.cs
│   ├── IRiddleService.cs
│   ├── IGameService.cs
│   └── IPowerUpService.cs
│
└── Services/
    ├── PlayerSessionService.cs
    ├── RiddleService.cs
    ├── GameService.cs
    └── PowerUpService.cs
```

---

### 📦 DTOs (Data Transfer Objects)

Los DTOs son objetos **planos** (sin lógica) para transferir datos entre capas.

**PlayerSessionDTO**

csharp

```csharp
public class PlayerSessionDTO
{
    public int Id { get; set; }
    public Guid SessionId { get; set; }
    public int TotalScore { get; set; }
    public int RiddlesSolved { get; set; }
    public IQLevel CurrentIQLevel { get; set; }
    public string IQLevelDescription { get; set; } // "Principiante", "Genio", etc.
    public DateTime CreatedAt { get; set; }
    public DateTime LastActivity { get; set; }
}
```

**RiddleDTO**

csharp

```csharp
public class RiddleDTO
{
    public int Id { get; set; }
    public string Question { get; set; }
    public string CorrectAnswer { get; set; }     // ⚠️ NO enviar al cliente
    public List<string> Options { get; set; }     // Opciones múltiples
    public string Hint { get; set; }
    public DifficultyLevel Difficulty { get; set; }
    public RiddleType Type { get; set; }
    public int TimeLimit { get; set; }
    public int BasePoints { get; set; }
}
```

**SubmitAnswerRequest**

csharp

```
public class SubmitAnswerRequest
{
    public int RiddleId { get; set; }
    public string Answer { get; set; }
    public int TimeSpent { get; set; }
    public bool UsedFiftyFifty { get; set; }
    public bool UsedHint { get; set; }
    public bool UsedExtraTime { get; set; }
}
```

**SubmitAnswerResponse**

csharp

```
public class SubmitAnswerResponse
{
    public bool IsCorrect { get; set; }
    public int PointsEarned { get; set; }
    public int BonusPoints { get; set; }
    public int TotalPoints { get; set; }
    public string CorrectAnswer { get; set; }
    public string Message { get; set; }
    public PlayerSessionDTO UpdatedSession { get; set; }
}
```

---

### 🔧 Servicios Implementados

Todos los servicios contienen la **lógica de negocio** del juego.

---

**1. IPlayerSessionService**

**Responsabilidad:** Gestionar sesiones de jugadores y niveles de IQ.

**Métodos:**

csharp

PlayerSessionDTO GetOrCreateSession(Guid sessionId);

PlayerSessionDTO GetSessionByGuid(Guid sessionId);

void UpdateSessionScore(int sessionId, int points);

void UpdateIQLevel(int sessionId);

` ` `

**Lógica de IQ Level:**

` ` `

< 300 pts → Principiante

300-600 pts → Promedio

600-1000 pts → Sobre Promedio

1000-1500 pts → Inteligente

1500-2500 pts → Muy Inteligente

2500-4000 pts → Genio

4000+ pts → Nivel Einstein

---

**2. IRiddleService**

**Responsabilidad:** Gestionar acertijos y selección aleatoria.

**Métodos:**

csharp

RiddleDTO GetRiddleById(int id);

RiddleDTO GetRandomRiddle(DifficultyLevel difficulty, int playerSessionId);

```
List<RiddleDTO> GetRiddlesByDifficulty(DifficultyLevel difficulty);
```

**Comportamiento de GetRandomRiddle:**

1. Intenta obtener un riddle NO intentado por el jugador
2. Si todos fueron intentados, devuelve uno aleatorio
3. Nunca devuelve CorrectAnswer al cliente (seguridad)

---

**3. IGameService**

**Responsabilidad:** Procesar respuestas y calcular puntuaciones.

**Métodos:**

csharp

```
SubmitAnswerResponse SubmitAnswer(Guid sessionId, SubmitAnswerRequest request);

List<GameAttemptDTO> GetPlayerHistory(Guid sessionId);

bool CanUseRiddle(Guid sessionId, int riddleId);
```

**Lógica de SubmitAnswer:**

1. Verifica respuesta (case-insensitive, trim)
2. Si es correcta:
    - Asigna puntos base según dificultad
    - Calcula bonus por rapidez (máx 50 pts)
    - Actualiza score del jugador
    - Actualiza IQ level si cambió
3. Registra el intento en BD
4. Devuelve resultado completo

**Cálculo de Bonus:**

csharp

```
bonus = (timeLimit - timeSpent) / timeLimit * 50
```

bonus = Math.Min(bonus, 50)

Ejemplo:

- TimeLimit: 60s, TimeSpent: 30s → Bonus: 25 pts
- TimeLimit: 60s, TimeSpent: 50s → Bonus: 8 pts
- TimeLimit: 60s, TimeSpent: 70s → Bonus: 0 pts

---

**4. IPowerUpService**

**Responsabilidad:** Gestionar power-ups del jugador.

**Métodos:**

csharp

```
List<PowerUpDTO> GetPlayerPowerUps(Guid sessionId);

bool UsePowerUp(Guid sessionId, PowerUpType powerUpType);

void InitializePowerUps(int playerSessionId);

int GetPowerUpCount(Guid sessionId, PowerUpType powerUpType);
```

**Power-ups disponibles:**

1. **50/50** - Elimina 2 opciones incorrectas
2. **Pista** - Muestra el hint del riddle
3. **Tiempo Extra** - Agrega tiempo al cronómetro

**Inicialización:** Cada jugador nuevo recibe **3 de cada power-up**.

---

🔌 **Inyección de Dependencias (Autofac)**

**Configuración en AutofacConfig.cs**

csharp

```
// DbContext

builder.RegisterType<GameDbContext>()

  .AsSelf()
```

```
    .InstancePerRequest();

// Repositorios
builder.RegisterType<PlayerSessionRepository>()
    .As<IPlayerSessionRepository>()
    .InstancePerRequest();
// ... (otros repositorios)

// Servicios
builder.RegisterType<PlayerSessionService>()
    .As<IPlayerSessionService>()
    .InstancePerRequest();
// ... (otros servicios)
```

**¿Cómo usarlo en Controllers?**

csharp

```
public class GameController : Controller
{
    private readonly IGameService _gameService;
    private readonly IRiddleService _riddleService;

    // ⭐ Autofac inyecta automáticamente
    public GameController(
        IGameService gameService,
        IRiddleService riddleService)
    {
        _gameService = gameService;
```

```
        _riddleService = riddleService;
    }

    public ActionResult Play()
    {
        // Usar los servicios...
        return View();
    }
}
```

---

## 📝 Flujo de Juego Completo

### 1️⃣ **Usuario entra al juego**

```
Usuario → Controller → IPlayerSessionService.GetOrCreateSession(sessionId)
                → Crea sesión + Inicializa 3 power-ups de cada tipo
                → Devuelve PlayerSessionDTO
```

### 2️⃣ **Usuario solicita un acertijo**

```
Usuario → Controller → IRiddleService.GetRandomRiddle(difficulty, playerSessionId)
                → Obtiene riddle no intentado (sin respuesta correcta)
```

→ Devuelve RiddleDTO

```

### 3️⃣ **Usuario usa un power-up**

```
Usuario → Controller → IPowerUpService.UsePowerUp(sessionId, powerUpType)
        → Reduce AvailableCount en 1
        → Devuelve true/false
```

### 4️⃣ **Usuario envía respuesta**

```
Usuario → Controller → IGameService.SubmitAnswer(sessionId, request)
        → Verifica respuesta
        → Calcula puntos + bonus
        → Actualiza score + IQ level
        → Registra intento
        → Devuelve SubmitAnswerResponse
```

### 5️⃣ **Usuario ve historial**

```
Usuario → Controller → IGameService.GetPlayerHistory(sessionId)
        → Devuelve List<GameAttemptDTO>
```

---

## 🛠️ Guía para Desarrollar en FP.MVC
```

✅ **Lo que YA está listo para usar:**

1. **Todos los servicios de lógica de negocio**
   - IPlayerSessionService
   - IRiddleService
   - IGameService
   - IPowerUpService
2. **Todos los DTOs para transferir datos**
   - PlayerSessionDTO
   - RiddleDTO
   - SubmitAnswerRequest/Response
   - PowerUpDTO
3. **Base de datos con 45 riddles precargados**
   - 15 Principiante (60s, 100pts)
   - 15 Intermedio (45s, 200pts)
   - 15 Experto (30s, 300pts)
4. **Inyección de dependencias configurada**
   - Autofac listo en AutofacConfig.cs

---

🎯 **Lo que DEBEN desarrollar en MVC:**

**1. ViewModels (FP.MVC/Models/ViewModels/)**

csharp

```csharp
// Ejemplo: GamePlayViewModel.cs
public class GamePlayViewModel
{
    public PlayerSessionDTO PlayerSession { get; set; }
    public RiddleDTO CurrentRiddle { get; set; }
```

```csharp
    public List<PowerUpDTO> AvailablePowerUps { get; set; }
    public int TimeRemaining { get; set; }
}
```

**2. Controllers (FP.MVC/Controllers/)**

csharp

```csharp
// Ejemplo: GameController.cs
public class GameController : Controller
{
    private readonly IGameService _gameService;
    private readonly IRiddleService _riddleService;
    private readonly IPowerUpService _powerUpService;
    private readonly IPlayerSessionService _sessionService;

    public GameController(
        IGameService gameService,
        IRiddleService riddleService,
        IPowerUpService powerUpService,
        IPlayerSessionService sessionService)
    {
        _gameService = gameService;
        _riddleService = riddleService;
        _powerUpService = powerUpService;
        _sessionService = sessionService;
    }

    // ... Actions
```

}

**3. Views (FP.MVC/Views/)**

- Game/Index.cshtml - Página principal
- Game/Play.cshtml - Vista de juego
- Game/Result.cshtml - Resultado de respuesta
- Game/History.cshtml - Historial de intentos
- Shared/_Layout.cshtml - Layout principal

**4. JavaScript/AJAX para interactividad**

- Cronómetro regresivo
- Validación de respuestas
- Uso de power-ups
- Actualización de UI sin reload

---

🔒 **Reglas de Seguridad (IMPORTANTE)**

❌ **NUNCA enviar al cliente:**

1. Riddle.CorrectAnswer antes de que responda
2. Cualquier información de otros jugadores
3. Listado completo de riddles con respuestas

✅ **Validaciones obligatorias:**

1. Verificar que el SessionId existe antes de cada operación
2. Validar que el jugador no haya intentado el mismo riddle dos veces
3. Verificar que tenga power-ups disponibles antes de usarlos
4. Validar time spent (no puede ser negativo ni mayor a timeLimit + extraTime)

---

📊 **Ejemplos de Uso desde Controllers**

**Obtener/Crear Sesión**

csharp

```
Guid sessionId = GetOrCreateSessionId(); // Cookie o Session

var playerSession = _sessionService.GetOrCreateSession(sessionId);
```

**Obtener Riddle Aleatorio**

csharp

```
var riddle = _riddleService.GetRandomRiddle(DifficultyLevel.Beginner,
playerSession.Id);

// ⚠️ IMPORTANTE: No enviar riddle.CorrectAnswer al View
```

**Enviar Respuesta**

csharp

```
var request = new SubmitAnswerRequest

{

   RiddleId = model.RiddleId,

   Answer = model.PlayerAnswer,

   TimeSpent = model.TimeSpent,

   UsedFiftyFifty = model.UsedFiftyFifty,

   UsedHint = model.UsedHint,

   UsedExtraTime = model.UsedExtraTime

};


var response = _gameService.SubmitAnswer(sessionId, request);

// response contiene: IsCorrect, PointsEarned, BonusPoints, UpdatedSession
```

**Usar Power-Up**

csharp

```
bool success = _powerUpService.UsePowerUp(sessionId, PowerUpType.Hint);

if (success)
```

```
{
   // Mostrar hint en UI
}
else
{
   // Mostrar mensaje "No tienes power-ups disponibles"
}
```

---

### 🐛 Debugging y Logs

**Puntos clave para debuggear:**

1. **Verificar SessionId:**

csharp

```
Debug.WriteLine($"SessionId: {sessionId}");
```

2. **Verificar inyección de dependencias:**

csharp

```
if (_gameService == null)
   throw new Exception("IGameService no fue inyectado");
```

3. **Verificar datos devueltos:**

csharp

```
if (riddle == null)
   Debug.WriteLine("No hay riddles disponibles para este nivel");
```

---

### 📚 Recursos Adicionales

**Enums disponibles:**

csharp

```
// DifficultyLevel
```

Beginner = 1, Intermediate = 2, Expert = 3

*// RiddleType*

MultipleChoice = 1, FreeText = 2

*// PowerUpType*

FiftyFifty = 1, Hint = 2, ExtraTime = 3

*// IQLevel*

Beginner = 1, Average = 2, AboveAverage = 3,

Smart = 4, VerySmart = 5, Genius = 6, Einstein = 7

---

⚠️ **Restricciones del Proyecto**

**NO implementar (fuera del alcance):**

- ❌ Sistema de login/registro
- ❌ Ranking mundial
- ❌ Tienda de power-ups
- ❌ Multijugador
- ❌ IA enemiga
- ❌ Modificación de estructura de BD sin autorización

**SÍ implementar:**

- ✅ Gestión de sesión con GUID (Cookie/Session)
- ✅ UI del juego con cronómetro
- ✅ Sistema de power-ups (3 iniciales de cada tipo)
- ✅ Historial de intentos del jugador

- ✅ Medidor de IQ visual

---

### 📞 Contacto y Dudas

**Si tienen dudas sobre:**

- Cómo usar un servicio específico → Revisar interfaces en FP.CORE/Interfaces/
- Estructura de datos → Revisar DTOs en FP.CORE/DTOs/
- Relaciones de BD → Revisar entidades en FP.DATA/Entities/
- Enums y sus valores → Revisar FP.DATA/Enums/

**Antes de modificar FP.CORE o FP.DATA:**

- ⚠️ Consultar primero
- ⚠️ No cambiar interfaces existentes
- ⚠️ No modificar estructura de BD

---

### ✅ Checklist para Empezar

- Referencias de proyectos configuradas correctamente
- Autofac instalado y configurado
- Base de datos creada y con datos
- Connection string en Web.config correcto
- Proyecto compila sin errores
- Servicios se inyectan correctamente en controllers
- Puedo obtener un riddle desde un controller de prueba